

===== SLIDE 1 =====

BANK PROJECT - COMPREHENSIVE ANALYSIS REPORT

===== BANK
PROJECT - COMPREHENSIVE ANALYSIS REPORT
=====

PROJECT OVERVIEW: A Java-based banking system that allows users to create accounts, login, and manage their finances including deposits, withdrawals, loans, and account details.

=====

===== SLIDE 2 =====

1. PACKAGES USED

=====

Core Java Packages: • java.text.NumberFormat - For currency formatting (US Dollar) • java.util.Locale - Locale-specific formatting (US) • java.util.Scanner - Console input handling • java.util.HashMap - Data structure for storing users • java.io.FileWriter - Writing user data to file • java.io.BufferedReader - Reading user data from file • java.io.FileReader - Opening the users.txt file

Custom Packages: • pages - UI screen displays (Pages.java) • ui - UI utilities (UI.java) • users - User management (Login.java, Creating.java, UsersStorage.java)

=====

===== SLIDE 3 =====

1. PROJECT MECHANICS & ARCHITECTURE

=====

The system operates in two main loops:

A) LOGIN LOOP (Initial):

- User selects: Login, Create Account, or Exit
- Validates credentials and returns username
- Loop breaks only after successful login
- Uses switch-case for menu navigation

B) USER HUB LOOP (After Login):

- Provides 8 operations: Details, Balance, Loan, Repay, Withdraw, Deposit, Transfer, Exit
- Continuously runs until user selects Exit (case 8)
- Uses switch-case for transaction routing

Data Persistence:

- Users stored in memory (HashMap) and persisted to "users.txt"
- Each user line format: username,password,accountType,mobile,balance,loan
- Loads from file on startup, saves after account creation or balance updates

=====

===== SLIDE 4 =====

1. HOW UI IS USED

=====

The UI class provides 2 static utility methods used throughout the application:

1. clearScreen():
2. Clears console for clean, organized display
3. Platform-aware: Uses "cls" for Windows, ANSI escape codes for Linux/Mac
4. ANSI codes: \033[H (cursor to top-left), \033[2J (clear screen)
5. Called before displaying each menu/transaction screen
6. pause(Scanner sc):
7. Waits for user to press Enter before continuing
8. Auto-clears screen after pause completes
9. Used between all major menu transitions

Color Formatting (ANSI Codes): • \033[32m - Green text (success messages) • \033[31m - Red text (error messages) • \033[34m - Blue text (balance display) • \033[1m - Bold text (emphasis) • \033[0m - Reset formatting

=====

===== SLIDE 5 =====

1. CLASSES, METHODS & FUNCTIONALITY

CLASS 1: Main.java

| Method: main(String[] args) | | — Entry point of application | | — Initializes Scanner, NumberFormat, Pages, UsersStorage objects | | — Manages both LOGIN LOOP and USER HUB LOOP | | — Handles 8 user operations: view details, check balance, take loan, etc. | | — Uses try-with-resources for automatic Scanner closure | | — User operations trigger saveToFile() after balance/loan updates |

CLASS 2: Pages.java

| Methods: (5 total) | | — homeScreen() : Displays Login/Create/Exit menu | | — loginScreen() : Shows "LOGGING IN" header | | — createScreen() : Shows "CREATING ACCOUNT" header | | — userHub() : Displays 8 user transaction options | | — loanRulesScreen() : Lists 6 loan rules and constraints | | | | Purpose: Handles all console UI screen layouts in one place | | Uses ANSI color formatting for visual appeal |

=====

===== SLIDE 6 =====

CLASS 3: UI.java

| Methods: (2 total - both static) | | — clearScreen() : Clears console screen (platform-aware) | | — pause(Scanner sc) : Waits for Enter key, then clears screen | | | | Purpose: Provides reusable UI utilities | | Impact: Improves UX with clean screen transitions |

CLASS 4: Login.java

| Method: handleLogin(Scanner sc, UsersStorage accountCheck) | | — Prompts user for username and password | | — Calls accountCheck.login() to validate credentials | | — Returns username if successful, null if failed | | — Displays colored error message on failure | | | | Purpose: Encapsulates login logic, called from Main's LOGIN LOOP |

=====

===== SLIDE 7 =====

CLASS 5: Creating.java

|

| Method: handleCreate(Scanner sc, UsersStorage accountCheck) | | — Prompts for: username, password, account type, mobile number | | — Validates each input (no empty fields, valid account type, digits only) | | — Regex validation: mobile.matches("\\d+") ensures numeric-only input | | — Calls accountCheck.createUser() to add user | | — Auto-saves to file if creation succeeds | | | Purpose: Handles new account creation with validations | | Called from Main's LOGIN LOOP (case 2) |

CLASS 6: UsersStorage.java (Core Data Management)

| INNER CLASS: User | | | Fields: username, password, accountType, mobile, balance, loan | | | Constructor: User(username, password, accountType, mobile, balance, loan) | | | MAIN CLASS: UsersStorage | | | Field: HashMap<String, User> users - In-memory user database | | | | | Constructor: UsersStorage() | | | | Calls loadFromFile() on initialization | | | | | PUBLIC METHODS: (9 total) | | | 1. saveToFile() : Writes all users to users.txt | | | 2. loadFromFile() : Reads users.txt and populates HashMap | | | 3. login() : Authenticates user credentials | | | 4. balance() : Returns user's current balance | | | 5. loan() : Returns user's current loan amount | | | 6. updateBalance() : Modifies user's balance | | | 7. updateLoan() : Modifies user's loan amount | | | 8. getUser() : Retrieves User object by username | | | 9. createUser() : Adds new user to HashMap | | | Purpose: Central data manager - all user data flows through this class |

=====

===== SLIDE 8 =====

1. TOTAL METHOD COUNT

=====

Main.java : 1 method Pages.java : 5 methods UI.java : 2 methods Login.java : 1 method Creating.java : 1 method UsersStorage.java : 9 methods (+ 1 inner class constructor) ————— TOTAL : 19 methods

(Not counting class constructors)

=====

===== SLIDE 9 =====

1. FILEWRITER USAGE & FILE PERSISTENCE

=====

FileWriter is used in UsersStorage.saveToFile() method:

Location: UsersStorage.java, saveToFile() method

Implementation: |— try-with-resources: try (FileWriter fw = new FileWriter("users.txt")) |— Overwrites entire "users.txt" file on each save |— Iterates through HashMap: for (User u : users.values()) |— Format: username,password,accountType,mobile,balance,loan\n |— Each user written as single comma-separated line

When saveToFile() is called: • After successful account creation (Creating.handleCreate) • After taking a loan (Main.java, case 3) • After repaying loan (Main.java, case 4) • After withdrawal (Main.java, case 5) • After deposit (Main.java, case 6)

Error Handling: • Catches IOException • Displays green success message: "Data saved successfully." • Displays red error message: "Error saving file."

Data Loading: • BufferedReader + FileReader used in loadFromFile() • Reads line-by-line with br.readLine() • Splits by comma: String data[] = line.split(",") • Validates line has 6 fields before adding to HashMap • Double.parseDouble() converts balance and loan back to numeric

=====

===== SLIDE 10 =====

1. KEY FEATURES & CONSTRAINTS

=====

Account Features: ✓ Multiple account types: personal, saving, rich, wealthy ✓ Account details: username, password, mobile, balance, loan ✓ All data persisted to disk

Transaction Features: 1. Check Balance : Displays current balance with currency formatting 2. Take Loan : Validates \$100-\$50,000 range, durations: 3/6/12/24 months 3. Repay Loan : Allows partial/full loan repayment 4. Withdraw : Validates amount < balance, maintains \$100 minimum 5. Deposit : Adds amount to balance 6. View Details : Shows username, mobile, account type, balance, loan 7. Transfer : Placeholder for future feature

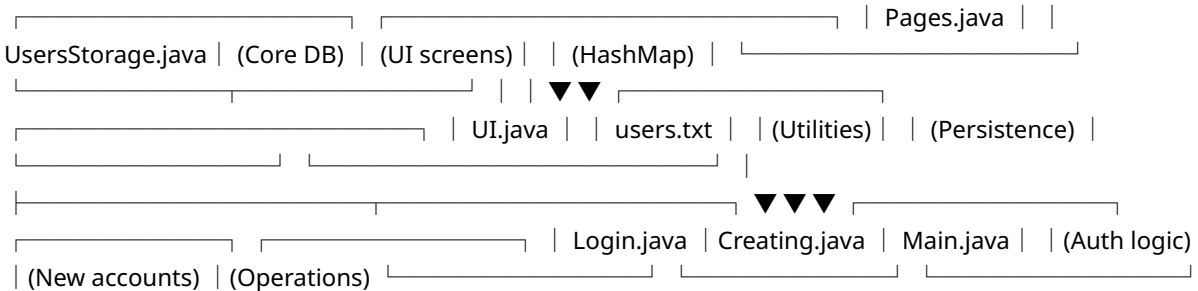
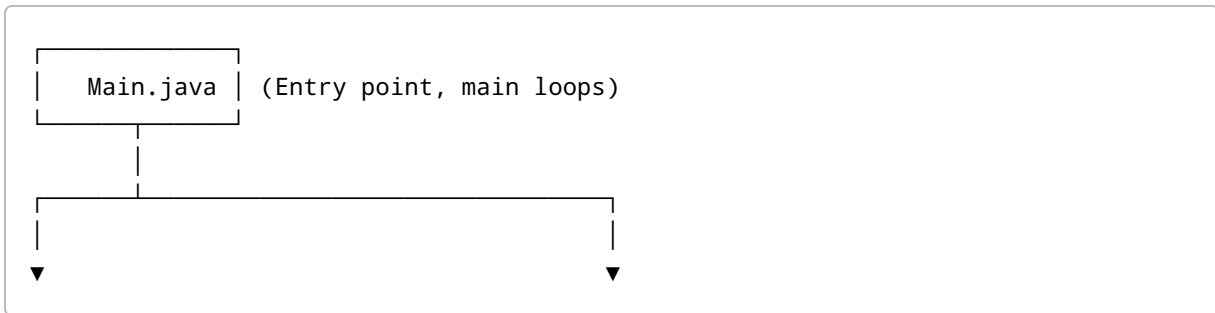
Validations: • Username: Cannot be empty, must be unique • Password: Cannot be empty • Mobile: Must contain digits only, no empty values • Account Type: Only "personal", "saving", "rich", "wealthy" allowed • Loan Amount: \$100 min, \$50,000 max per rules screen • Withdrawal: Cannot exceed balance, minimum \$100 must remain • Deposit/Repay: No negative amounts allowed

=====

===== SLIDE 11 =====

1. DATA FLOW SUMMARY

=====



=====

===== SLIDE 12 =====

1. TECHNOLOGY STACK

=====

Language: Java Data Structure: `HashMap<String, User>` File Format: Text (CSV-like: comma-separated values) Input: Scanner (console) Output: System.out (console with ANSI formatting) Architecture: Procedural with utility classes

=====

===== SLIDE 13 =====

1. PROJECT STRENGTHS & POTENTIAL IMPROVEMENTS

=====

Strengths: ✓ Clear separation of concerns (Login, Creating, UsersStorage, Pages, UI) ✓ Persistent data storage (FileWriter/BufferedReader) ✓ Input validation with regex ✓ User-friendly with colored output and pauses ✓ Error handling for file I/O and invalid input ✓ Try-with-resources for resource management

Potential Improvements: • Transfer functionality (currently placeholder) • Transaction history/logging • Interest calculation for loans • Database instead of text file • Password encryption (currently plain text) • User session management • More robust error handling • Deposit/Withdrawal history tracking

=====

===== SLIDE 14 =====

END OF REPORT

===== END OF
REPORT =====